

NutriGuard AI — Problem Statement and Project Type

Project Name

NutriGuard AI: Event-Driven Multi-Agent Health & Nutrition Tracker

1. User and Context

The target users are health-conscious people, working professionals, students, and fitness beginners who want to improve their diet but do not track meals in detail.

Many users only remember meals in simple language, such as:

I ate poha with tea.
I had dal rice and sabzi.
I missed my iron tablet.

Some users may also have basic health conditions or nutrition concerns such as:

- Iron deficiency
 - Vitamin D deficiency
 - B12 deficiency
 - Fat loss goal
 - Muscle gain goal
 - Vegetarian diet
-

2. Need / Pain

Most users do not know whether their daily meals match their health goal or health condition.

For example, a user with iron deficiency may drink tea with meals without knowing that it may need attention. A user trying fat loss may eat mostly carbohydrate-heavy meals and miss protein.

Current problems:

- Users do not log exact calories or macros.

- Normal diet apps require too much manual input.
 - Generic AI advice may not be grounded in trusted nutrition knowledge.
 - Users need simple risk flags, not complex medical advice.
 - Users need safe suggestions without diagnosis or medicine dosage.
-

3. Proposed Solution

NutriGuard AI is a multi-agent GenAI system that reads a user's meal log, checks it against their health goal and health condition, and generates safe nutrition flags and simple recommendations.

The system uses 3 agents:

1. **Meal Analyzer Agent**
Understands the user's meal and extracts structured food information.
 2. **Health Risk & Supplement Agent**
Checks the meal against the user's goal, condition, and supplements using Azure AI Search and Gemini.
 3. **Report Agent**
Creates a simple daily nutrition report with suggestions and safety notes.
-

4. Success Measure

The project will be successful if it can:

- Correctly extract food items from simple meal text.
- Correctly identify basic nutrition risks.
- Generate safe and useful recommendations.
- Avoid diagnosis, medicine dosage, or unsafe supplement advice.
- Complete report generation within a reasonable time.
- Track the full request using centralized logging and correlation ID.

Success metrics:

- Food extraction accuracy: above 90%
- Risk flag accuracy: above 85%
- JSON validity rate: above 95%
- Safety violation rate: 0%
- End-to-end latency: under 20 seconds for demo
- Failed workflow rate: below 5%

5. Data Source

The system will use both user-provided data and curated nutrition knowledge.

User Data

- User profile
- Health goal
- Diet type
- Health conditions
- Supplements
- Daily meal text

Knowledge Base Data

Stored in Azure AI Search:

- Iron deficiency guidance
- Tea and iron absorption notes
- Vegetarian protein sources
- Fat loss nutrition basics
- Vitamin B12 notes
- Vitamin D notes
- Supplement safety notes

System Data

Stored in PostgreSQL:

- Users
 - User profiles
 - Meal logs
 - Nutrition flags
 - Daily reports
 - Outbox events
-

6. Scope Fence

For the assignment MVP, NutriGuard AI will only focus on:

- Simple meal text input
- Basic health goal and condition tracking
- 3-agent workflow
- Safe risk flags
- Daily nutrition report
- Event-driven processing
- Local development using PostgreSQL
- Azure-ready architecture

Out of scope for MVP:

- Exact calorie calculation
 - Medical diagnosis
 - Medicine dosage recommendation
 - Prescription changes
 - Image-based food recognition
 - Wearable device integration
 - Full mobile app
 - Real doctor consultation workflow
-

Final Problem Statement

For **health-conscious users with nutrition goals or basic health conditions**, who need **simple and safe feedback on daily meals without manually tracking calories**, we build **NutriGuard AI**, a **multi-agent nutrition risk and recommendation assistant** that provides **meal analysis, risk flags, and daily improvement suggestions**, measured by **risk flag accuracy, JSON validity, safety compliance, and report generation time**.

Unlike **generic calorie trackers or ungrounded chatbot advice**, NutriGuard AI uses agents to **reason over meal logs, retrieve trusted nutrition knowledge, and act by generating safe personalized reports**.

Step 2: Choose Project Type

Selected Project Type

Primary Project Type: Recommender

NutriGuard AI is mainly a **recommender system** because the final output suggests the next best action for the user.

Example recommendations:

- Add protein to breakfast.
 - Avoid tea close to iron-rich meals.
 - Add lemon or amla for vitamin C support.
 - Replace biscuits with chana, sprouts, curd, or fruit.
 - Follow doctor advice for supplements.
-

Why Recommender is the Best Fit

The main purpose of NutriGuard AI is not only to answer questions. Its main purpose is to analyze the user's meal and recommend safer, better choices.

The system takes:

- Meal text
- User goal
- Health condition
- Supplement routine
- Nutrition knowledge

Then it generates:

- Risk flags
- Improvement suggestions
- Daily nutrition report

So the best project category is:

Recommender — Suggest the next best item or action

Agentic Angle

NutriGuard AI has a strong agentic angle because it uses multiple agents to complete different tasks.

Agent 1: Meal Analyzer Agent

Reasoning task:

- Understand meal text
- Extract food items
- Classify food categories

Example:

Poha with tea

Output:

```
{  
  "foods": ["poha", "tea"],  
  "carb_sources": ["poha"],  
  "protein_sources": [],  
  "beverages": ["tea"]  
}
```

Agent 2: Health Risk & Supplement Agent

Reasoning + retrieval task:

- Check user goal
- Check health condition
- Retrieve trusted knowledge from Azure AI Search
- Generate risk flags

Example:

Condition: Iron deficiency

Meal: Poha with tea

Output:

```
{  
  "risk_flags": [  
    {  
      "type": "iron_attention",  
      "severity": "medium",  
      "message": "Tea near meals may need attention for iron deficiency."  
    }  
  ]  
}
```

```
]
}
```

Agent 3: Report Agent

Acting task:

- Create final daily report
- Convert risk flags into simple language
- Add safe suggestions
- Add safety disclaimer

Example output:

Your breakfast was mostly carbohydrate-based.

Needs attention:

- Breakfast has low protein.
- Tea near meals may not be ideal for iron deficiency.

Suggestion:

Add sprouts, curd, paneer, tofu, chana, or soy chunks.

Safety:

Follow your doctor's advice for supplements.

Secondary Project Types

NutriGuard AI also has small parts of:

Chatbot

Because the user may enter meals in natural language.

Example:

I ate poha with tea and missed my iron tablet.

Summariser

Because the Report Agent summarizes the daily nutrition pattern.

Example:

Today's meals were mostly carbohydrate-heavy with low breakfast protein.

But the main project type remains:

Recommender

Final Project Type Statement

NutriGuard AI is a **Recommender-type multi-agent GenAI project** that suggests safe nutrition improvements based on meal logs, health goals, and health conditions.

It uses:

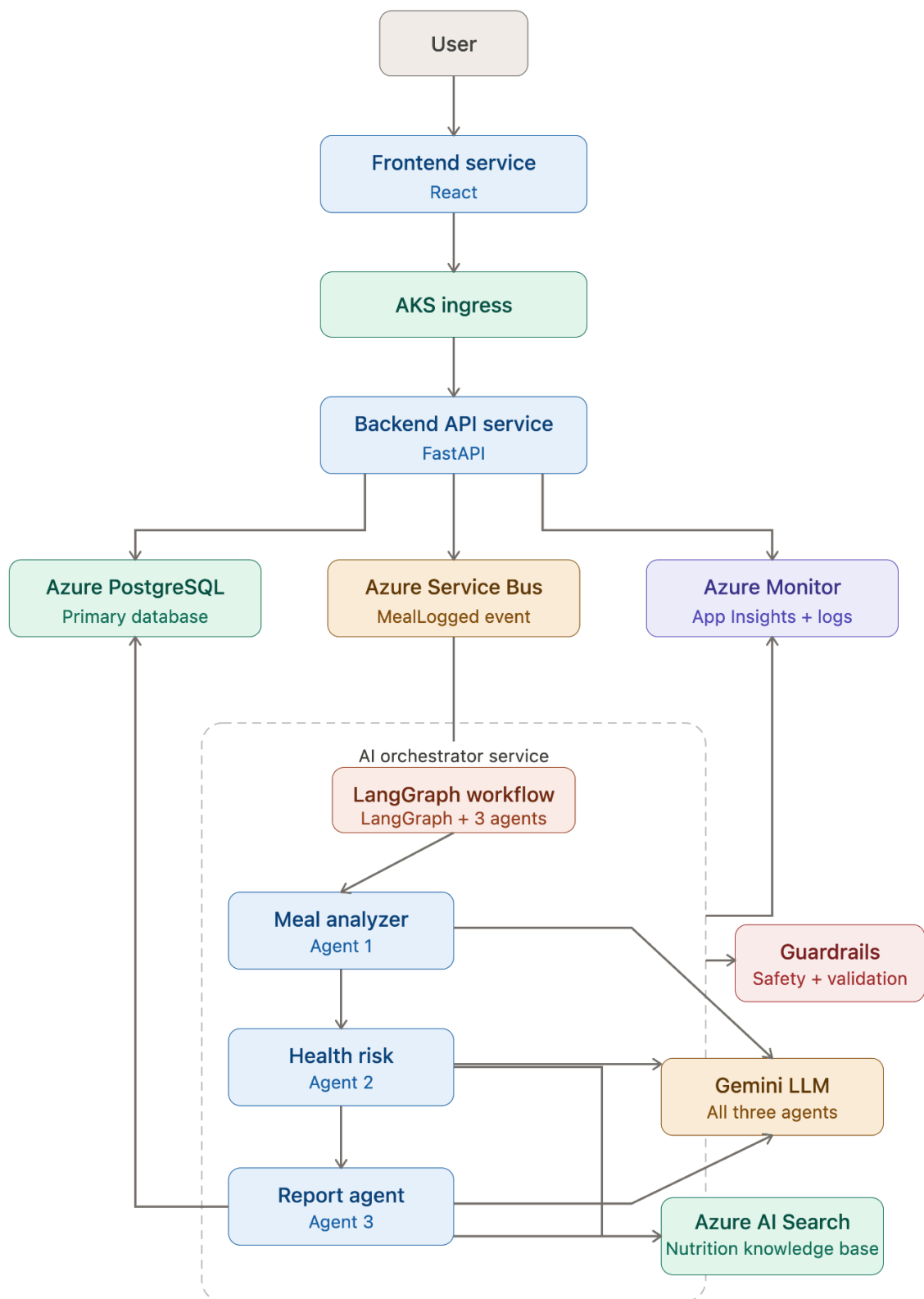
- **Reasoning** to understand meals and identify risks.
- **Retrieval** through Azure AI Search to use trusted nutrition knowledge.
- **Action** by generating daily reports and recommendations.

Therefore, it clearly maps to the **Recommender archetype** with a strong multi-agent architecture.

You can use this directly in your assignment document

NutriGuard AI — HLD, Tech Stack, and Azure Services

1. High-Level Architecture Diagram



Click any node to learn more [Application service](#) [Messaging / LLM](#) [Storage / search](#) [Orchestration](#)

2. Simple Architecture Explanation

NutriGuard AI has three main services:

1. **Frontend Service**
User enters meals, health goal, and health condition.
2. **Backend API Service**
Saves user data and meal logs, then publishes a **MealLogged** event.
3. **AI Orchestrator Service**
Contains LangGraph and 3 agents:
 - Meal Analyzer Agent
 - Health Risk Agent
 - Report Agent

The AI Orchestrator uses **Gemini** for generation and **Azure AI Search** to retrieve trusted nutrition knowledge.

3. Agent Flow

User meal input

↓

Meal Analyzer Agent

↓

Health Risk Agent

↓

Report Agent

↓

Final nutrition report

Example:

Input:

I ate poha with tea. I have iron deficiency.

Output:

- Breakfast is low in protein.
 - Tea near meals may need attention for iron deficiency.
 - Add sprouts, curd, paneer, tofu, chana, or soy chunks.
 - Follow doctor's advice for supplements.
-

4. Selected Tech Stack

Tech stack

Layer	Tool
Frontend	React
Backend API	FastAPI
Agent orchestration	LangGraph
Agents	Python
LLM	Gemini API
RAG / vector search	Azure AI Search
Database	PostgreSQL
Event / queue	Azure Service Bus
Containerization	Docker
Hosting	Azure Kubernetes Service
Ingress	AKS Ingress
CI/CD	GitHub Actions
Container registry	Azure Container Registry
Monitoring	Azure Monitor + Application Insights
Logs	Log Analytics Workspace
Secrets	Azure Key Vault
Guardrails	Safety prompts, JSON validation, Azure AI Content Safety (optional)

5. Azure Services Mapping

Requirement	Azure service
Host frontend, backend, and AI orchestrator	Azure Kubernetes Service
Route traffic to services	AKS Ingress
Store Docker images	Azure Container Registry
Store users, profiles, meals, reports	Azure Database for PostgreSQL
Event-driven communication	Azure Service Bus
RAG knowledge base	Azure AI Search
Store secrets / API keys	Azure Key Vault
CI/CD deployment	GitHub Actions
Centralized monitoring	Azure Monitor
Application tracing	Application Insights
Central log storage	Log Analytics Workspace
Dashboard	Azure Monitor Workbook
Autoscaling	KEDA on AKS
Guardrails	Prompt rules + validation + optional Azure AI Content Safety

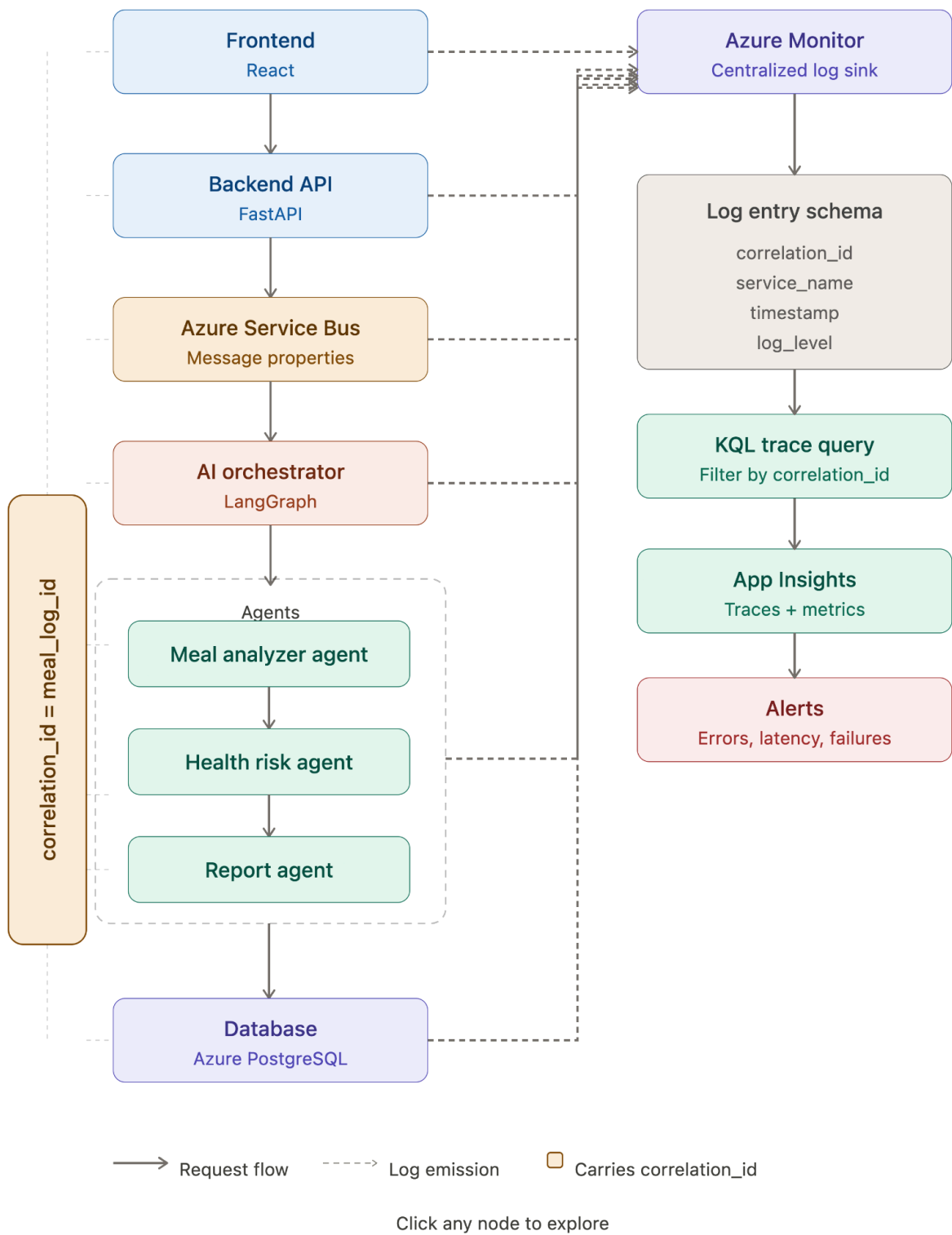
6. Observability Plan

Use centralized logging for all services.

Every request will have a common ID:

`correlation_id = meal_log_id`

This ID will be passed through:



Monitor:

- API latency
 - AI Orchestrator latency
 - Agent-wise latency
 - Service Bus queue depth
 - Failed events
 - Gemini errors
 - Azure AI Search errors
 - PostgreSQL errors
 - Pod restarts
 - End-to-end report generation time
-

7. Guardrails Plan

NutriGuard AI will use guardrails to keep the output safe.

Guardrails:

- No medical diagnosis
- No medicine dosage recommendation
- No supplement dose change
- Always add safety note for health conditions
- Validate JSON output from agents
- Block unsafe or overconfident medical claims
- Optional Azure AI Content Safety for input/output checks

Example safety note:

This is general nutrition guidance. Follow your doctor's advice for supplements or medical conditions.

8. Final One-Line Architecture Summary

NutriGuard AI is an event-driven multi-agent nutrition recommender where FastAPI publishes meal events to Azure Service Bus, a LangGraph-based AI Orchestrator runs 3 agents using

Gemini and Azure AI Search, PostgreSQL stores the results, and Azure Monitor provides centralized observability.